PROGETTO WEB / PRESENTAZIONE TECNICA

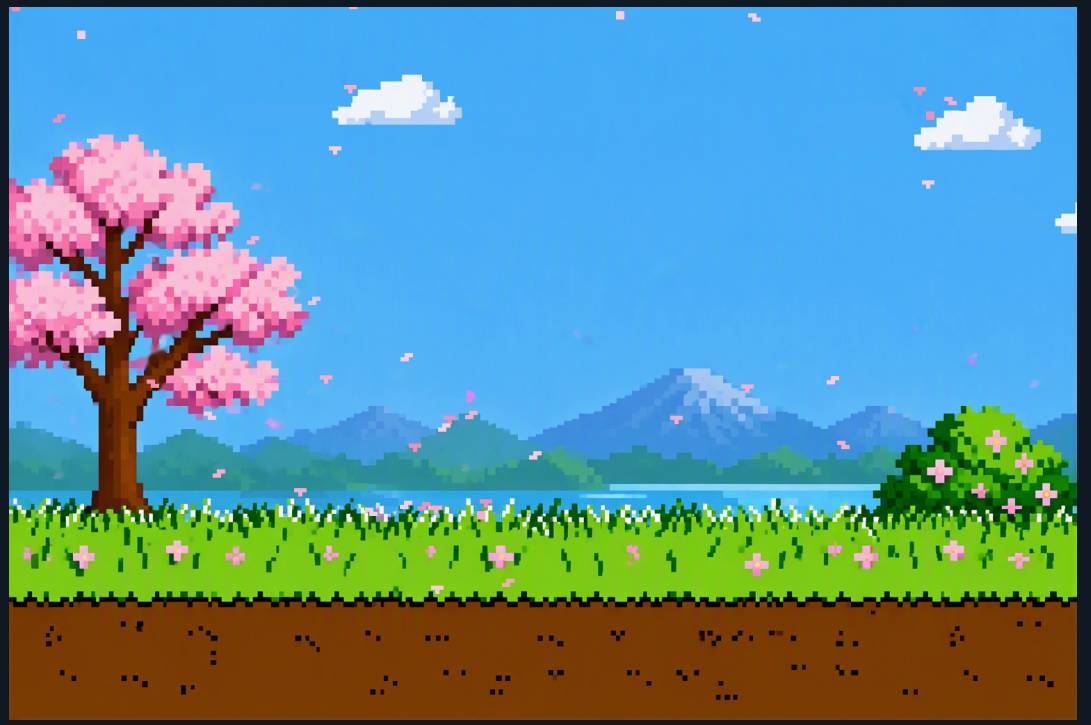
DUCK HUNT NL

Un gioco arcade in pixel art, costruito con HTML, CSS e JavaScript puro.

Varisco Nicolò
Androni Lorenzo

5 round · 50 bersagli · 3 colpi per anatra

Dall’idea alle scelte progettuali,
fino al funzionamento del codice.



La homepage: si parte da qui

Nome del progetto, scenario animato e collegamento diretto alla partita.



Identità del progetto

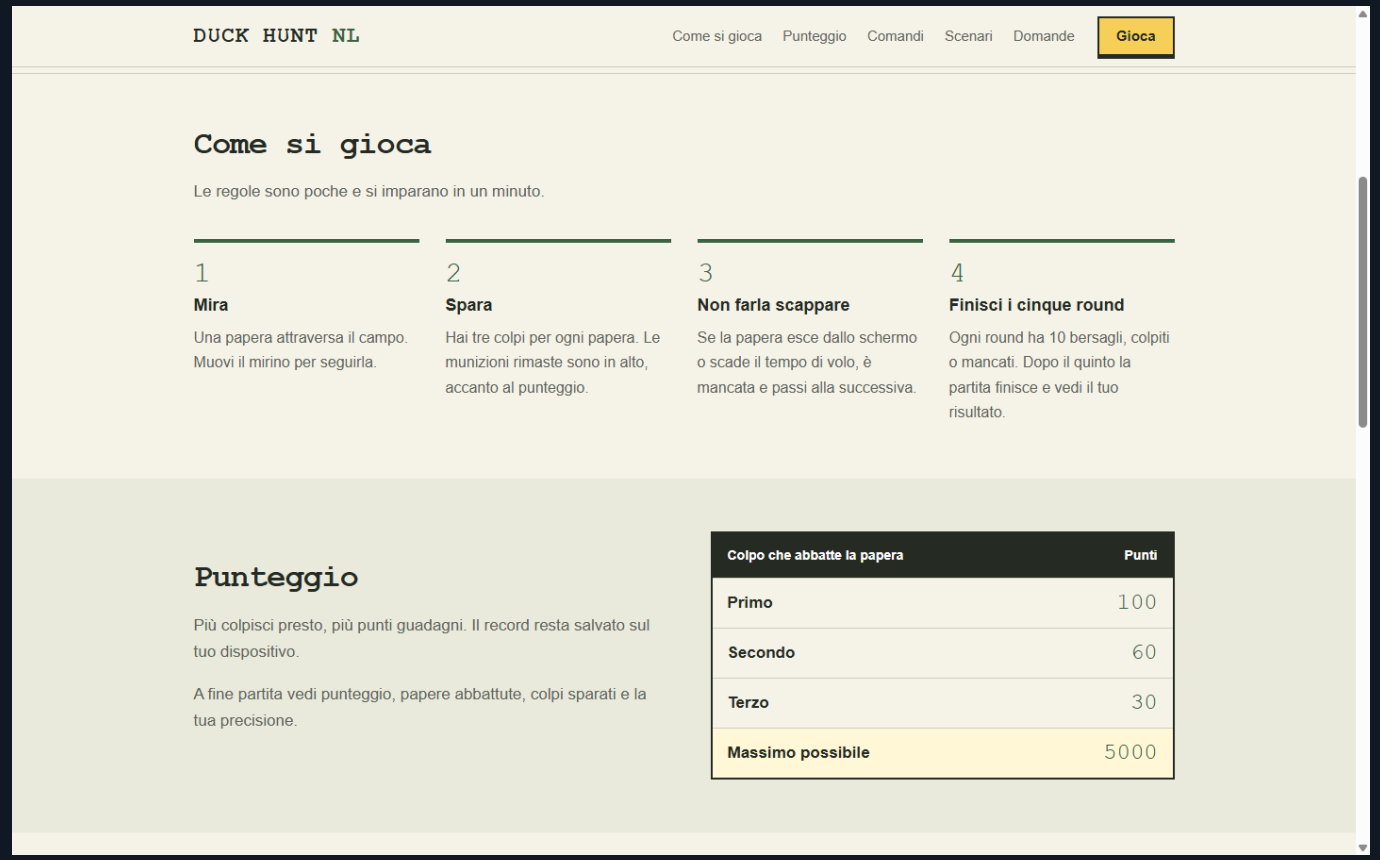
Titolo semplice e grafica ispirata al gioco originale.

Due scelte chiare

Gioca ora apre gioco.html; Leggi le regole porta alle istruzioni.

Prima di giocare: regole e punteggio

La pagina spiega cosa fare e come vengono assegnati i punti.



Regole in quattro passi

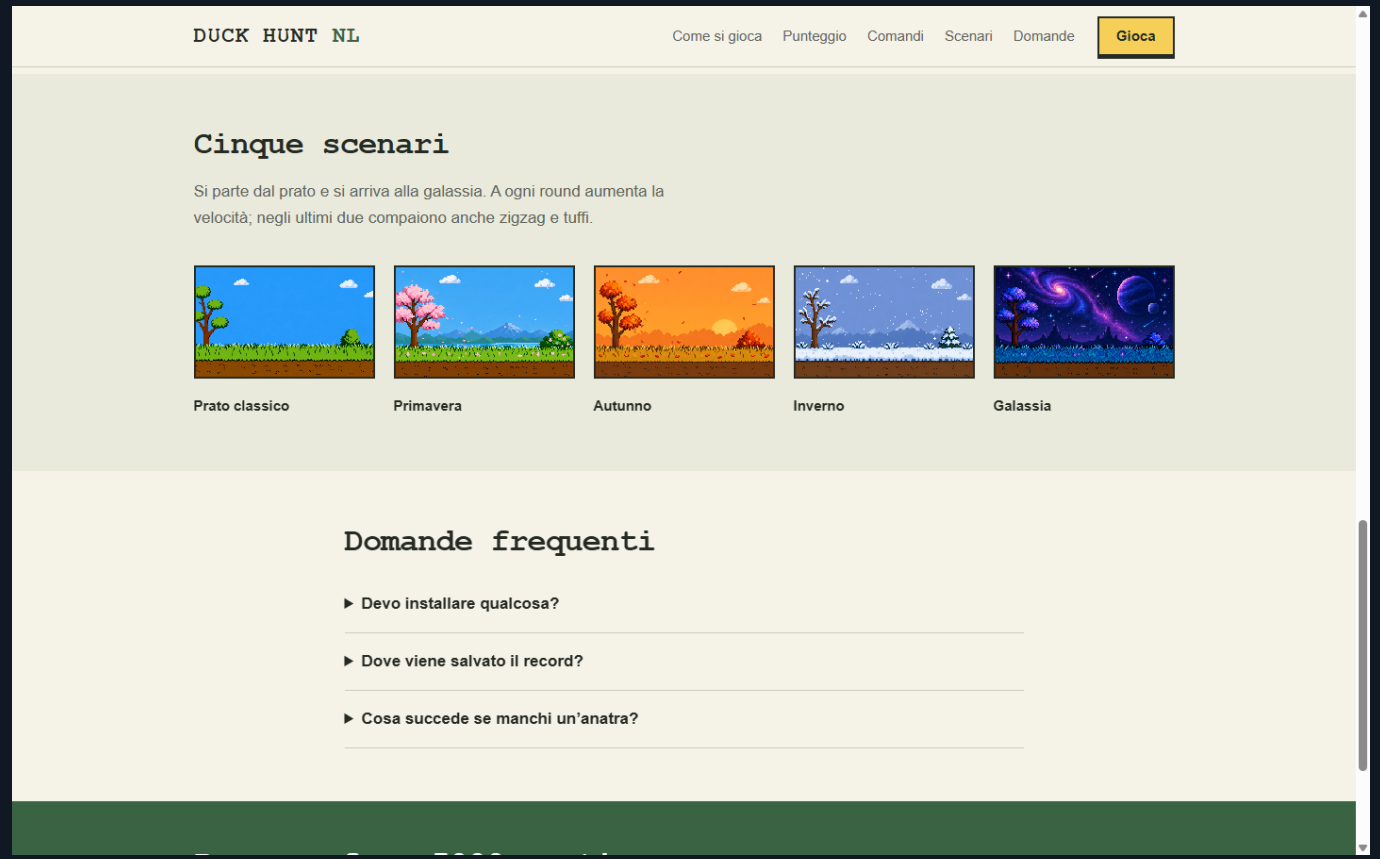
Mira, spara, non far scappare l’anatra e completa i round.

Punti e record

La tabella confronta i tentativi: 100, 60 e 30 punti.

Scenari e domande frequenti

L’utente vede le ambientazioni e trova risposte prima di iniziare.



Cinque ambientazioni

Le anteprime video mostrano gli scenari dei round.

Informazioni pratiche

Installazione, record locale e bersagli mancati: risposte brevi e dirette.

Le regole diventano interazione

Funzionalità presenti nella versione del progetto analizzata.

Partita completa

5 round × 10 bersagli; 3 colpi per anatra; 100 / 60 / 30 punti.

Comandi e riscontri

Mouse per mirare; clic o Spazio per sparare; P / Esc per la pausa.

Progressione e risultati

Cinque scenari, traiettorie variabili, statistiche finali e record locale.

Responsabilità separate, file collegati

HTML organizza i contenuti, CSS definisce l’aspetto, JavaScript governa la partita.

Pagine HTML

index.html: sito informativo → gioco.html: interfaccia e campo di gioco.

Fogli di stile

sito.css: sito principale / style.css: gioco, layout e sovrapposizioni.

Logica e risorse

script.js: stato, eventi e regole / img e gif: immagini, GIF e video MP4.

Tre livelli compongono il campo

La scena combina elementi DOM e disegno su canvas.



01 VIDEO MP4

Paesaggio animato sullo sfondo.

02 IMMAGINE GIF

Anatra in volo e in caduta, spostata tramite stili CSS.

03 CANVAS + HTML

Mirino ed effetti sul canvas; punteggi e pannelli nell'HTML.

Lo stato: una sola fonte per la partita

Estratto dal codice attuale; le omissioni sono indicate con // …

```
const statoGioco = {
    ...statoIniziale,
    colpitePerTurno: Array(NUMERO_TURNI).fill(0),
    fase: "pronta",
    recordPersonale: 0,
    recordDisponibile: true,
    mira: { x: LARGHEZZA / 2,
            y: ALTEZZA / 2, visibile: false },
    ultimoTempo: 0,
    gettonePartita: 0,
    graficaCaricata: false,
};
```

script.js / statoIniziale e statoGioco

COME FUNZIONA

Dati condivisi

Fase, mira e contatori sono raccolti nello stesso oggetto.

Reset controllato

statoIniziale contiene i valori da azzerare a ogni nuova partita.

Le fasi impediscono azioni fuori tempo

Estratto dal codice attuale; le omissioni sono indicate con // …

```
function lanciaAnatra() {
    statoGioco.fase = "inCorso";
    creaAnatra();
    nascondiOverlay();
    // …
}

function spara() {
    if (statoGioco.fase !== "inCorso" ||
        statoGioco.colpi === 0) return;
    // …
}
```

script.js / lanciaAnatra e inizio di spara

COME FUNZIONA

Sequenza principale

pronta → traRound → inCorso → traBersagli; poi nuovo bersaglio o fine.

Guardia iniziale

Si può sparare soltanto durante il volo, con munizioni disponibili.

Il game loop: aggiornare prima di disegnare

Estratto dal codice attuale; le omissioni sono indicate con // …

```
function aggiornaFotogramma(tempoAttuale) {
    const intervallo = Math.min(
        (tempoAttuale - statoGioco.ultimoTempo)
        / 1000, .05);
    statoGioco.ultimoTempo = tempoAttuale;
    if (statoGioco.fase !== "inPausa") {
        if (statoGioco.fase === "inCorso")
            aggiornaAnatra(intervallo);
        // … aggiornamento caduta ed effetti
    }
    disegna();
    requestAnimationFrame(aggiornaFotogramma);
}
```

script.js / aggiornaFotogramma

COME FUNZIONA

Delta time

La differenza fra timestamp diventa un intervallo in secondi.

Limite di 0,05 s

Riduce i salti dopo un frame lento. Il browser pianifica il frame seguente.

La hitbox decide se il colpo è a segno

Estratto dal codice attuale; le omissioni sono indicate con // …

```
statoGioco.colpi -= 1;
statoGioco.colpiSparati += 1;
const { anatra, mira } = statoGioco;
const scala = scalaGioco();
const colpita =
    Math.abs(mira.x - anatra.x) < 27 * scala
    && Math.abs(mira.y - anatra.y) < 21 * scala;
const numeroColpo =
    COLPI_PER_ANATRA - statoGioco.colpi;
const punti = colpita
    ? PUNTI_PER_COLPO[numeroColpo - 1] : 0;
```

script.js / spara

COME FUNZIONA

Collisione rettangolare

Entrambe le distanze dal centro devono rientrare nei limiti della hitbox.

Punti per tentativo

L’indice numeroColpo − 1 seleziona 100, 60 o 30; una mancata vale zero.

Asincrono: attendere senza bloccare il gioco

Estratto dal codice attuale; le omissioni sono indicate con // …

```
const attendi = (millisecondi) =>
    new Promise((completa) =>
        window.setTimeout(completa, millisecondi));

async function continuaPartita(
    gettone, messaggio, mostraCaduta = false) {
    statoGioco.fase = "traBersagli";
    statoGioco.anatra = null;
    statoGioco.bersagliTurno += 1;
    // …
    await attendi(mostraCaduta
        ? DURATA_CADUTA * 1000 : 900);
    // …
}
```

script.js / attendi e continuaPartita

COME FUNZIONA

Prima dell’await

La funzione cambia subito fase, rimuove l’anatra e conta il bersaglio.

Durante l’attesa

Si sospende questa funzione: il browser continua a disegnare e gestire eventi.

Il gettone evita che un’attesa vecchia interferisca

Estratto dal codice attuale; le omissioni sono indicate con // …

```
await attendi(mostraCaduta
    ? DURATA_CADUTA * 1000 : 900);
if (statoGioco.fase !== "traBersagli" ||
    statoGioco.gettonePartita !== gettone) return;
statoGioco.anatraInCaduta = null;

if (!turnoFinito) return lanciaAnatra();
if (statoGioco.turno === NUMERO_TURNI)
    return terminaPartita();
statoGioco.turno += 1;
statoGioco.bersagliTurno = 0;
await mostraIntroTurno(gettone);
```

script.js / seconda parte di continuaPartita

COME FUNZIONA

Controllo al rientro

Dopo l’attesa si verifica che fase e partita siano ancora quelle previste.

Tre percorsi

Altro bersaglio, fine partita oppure round successivo con una nuova intro.

Caricare la grafica con Promise.all

Estratto dal codice attuale; le omissioni sono indicate con // …

```
const caricaImmagine = (percorso) =>
  new Promise((completa, rifiuta) => {
    const immagine = new Image();
    immagine.onload = () => completa(immagine);
    immagine.onerror = () => rifiuta(
      new Error("Impossibile caricare " + percorso));
    immagine.src = percorso;
  });
// …
Promise.all([
    // … percorsi di GIF, mirini e cane
].map(caricaImmagine)).then((/* … */) => {
    // … abilita avvio dopo il caricamento
}).catch((errore) => { /* … mostra errore */ });
```

script.js / caricaImmagine e caricamento finale

COME FUNZIONA

Tutte pronte

map avvia i caricamenti; Promise.all si risolve quando tutti hanno successo.

Errore gestito

Se una risorsa fallisce, catch mostra il problema e l’avvio resta disabilitato.

Sincrono e asincrono nello stesso progetto

I callback eseguono istruzioni in sequenza; il browser decide quando richiamarli.

SINCRONO

spara(): controlli → munizioni
→ hitbox → punti.

aggiornaAnatra(): movimento
e timer nello stesso callback.

localStorage: lettura e scrittura
con gestione try/catch.

ASINCRONO

setTimeout + await:
ripresa dopo una pausa.

Image.onload + Promise.all:
attesa delle risorse grafiche.

requestAnimationFrame ed eventi:
callback richiamati dal browser.

Le criticità affrontate nel codice

Problemi tecnici e relative soluzioni osservabili nell’implementazione.

Attese pendenti e azioni fuori fase

Gettone di partita + controlli sulla fase dopo await e prima dello sparo.

Frame lenti e ridimensionamento

Delta time limitato; adattaCanvas riscala posizioni, velocità e risoluzione.

Risorse o salvataggio non disponibili

catch per le immagini; try/catch per localStorage; avvisi e avvio protetto.

Grazie per l’attenzione!

DUCK HUNT NL

Varisco Nicolò

Androni Lorenzo

Un sito per scoprire. Un gioco per mettersi alla prova.

Siamo pronti per le vostre domande.